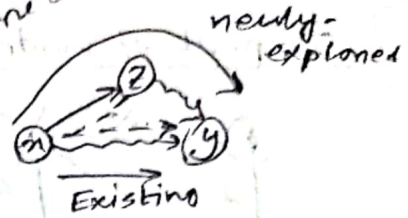
CSF-321

08/12/2025

DVR: Solution to Count-to-infinity problem

unit-level after receiving one entry from some table



not possible to reach

avoid paths where nexthop is the curr node (split the horizon)

update $(x, y, z) \Rightarrow$ $\boxed{\text{single update after an info is received}}$

$d \leftarrow c(x,z) + d(z,y)$

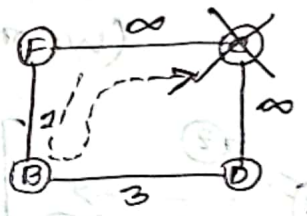
if $d < d(x,y)$ & $\underbrace{x \neq nexthop(z,y)}_{\text{Split Horizon Rule}}$

return d, z

else return $d(x,y), nexthop(x,y)$

$\Rightarrow$ Now fully ok? $\Rightarrow$ NO, as the entry of D's table still mistakenly gets updated.

Reason: B's stale table entry to C updates the table in D, as the algo only decreases the cost, however, it does not increase it



invalid loop → paradox

F→B→F→C

B's distance should be ∞ $\boxed{\text{forcefully}}$

$\Rightarrow$ curr algo only decreases

Soln: Always accomodate the update from the next-hop node

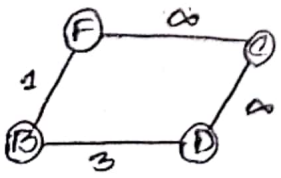
update (x, y, z)

$d \leftarrow c(x,z) + d(z,y)$

if $\underbrace{nexthop(x,y) = z}_{\text{Forced update}}$ | $(d < d(x,y)$ & $x \neq nexthop(z,y))$

return d, z → best path $\Rightarrow$ goes through next hop → must rely on its info.

else return $d(x,y), nexthop(x,y)$



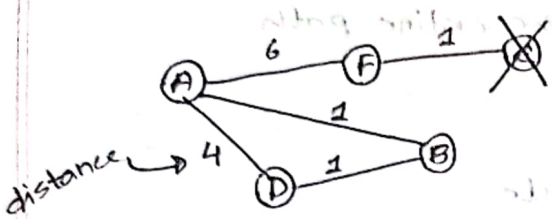
$\Rightarrow$ | C | 5 | B | in D $\Rightarrow$ | C | ∞ | – | in B $\Rightarrow$ | C | ∞ | – | in D (OK)

F's problem solved → split horizon rule

B's " " → Forced update

D's " " → Forced update

Ques: All problems are solved?



A	B	D	F
c | 7 | F	c | 8 | A	c | 9 | B	c | 1 | c
–	–	–	c | ∞ | – ⇒ c goes down
c | ∞ | –	–	–	–
–	c | ∞ | –	–	–
–	–	c | ∞ | –	–

forced update

↳ D not yet updated
it D's info reaches A at this point

arrival of D's entry to A

A	B	D	F	
c | 13 | D	–	–	–	
–	c | 14 | A	–	–	→ new update from A
–	–	c | 15 | B	–	→ new update from B
c | 19 | D	–	–	–	→ new update from D

⋮

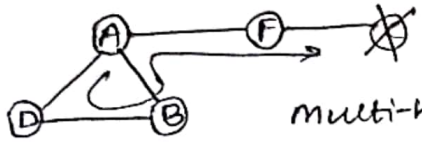
Count-to-infinity Problem!

next-hop = cum → cycle

not next-hop but on the path → still creates a biggen circle
multi-hop circle

Reason behind sequence-dependent convergence

→ Only the next-hop is known, not the entire path



Multi-hop circular route

Possible soln: Track down & distribute the entire path

overhead ↑ ⇒ Problem: lot more info to be xmitted

Other soln: Ensure better sequence in update

① Propagate bad news faster: Trigerred update for rev ∞

② Make infinity smaller finite # (e.g., 100) → sufficient to repr ∞

↳ none for a network to have 100 hops.

propagation sequence still may cause issues

Link State Routing (LSR):

DVR: sends info of all nodes to neighbors

LSR: sends info of neighbors to all nodes

↳ can construct the entire network

↳ full graph

↳ shortest path

Five operational steps in LSR

fundamental steps

① Discover neighbors

② Measure cost to each neighbor

③ Construct a pckt incorporating the cost to neighbors

↳ info unit (pckt)

④ Send the info to all other routers

⑤ Compute shortest path to every router

time delay, # hops, ...

good news → has intervals 5-10s

bad news → propagate instantly

nobody can arrive in-between